

LAPORAN TUGAS 1

SISTEM OPERASI



Dosen Pengampu Mata Kuliah:
Triawan Adi Cahyanto M.Kom

Disusun Oleh:
Mohamad Qowi Aizani (2410651003)

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

TUGAS 1 – PROCESS MANAGER

- **Pengertian**

Program ini merupakan implementasi **Process Manager** dengan menggunakan system call tingkat rendah, yaitu `fork()`, `pipe()`, `waitpid()`, dan `gettimeofday()`. Tujuan program adalah mendemonstrasikan:

- Pembuatan proses (parent dan child).
- Komunikasi antarproses menggunakan **pipe** dan **shared memory**.
- Sinkronisasi parent dengan child menggunakan `waitpid()`.
- Pengukuran waktu eksekusi tiap child process.

- **Deskripsi Program**

Program membuat **3 child process**, dengan tugas:

1. **Child 1:** Menghitung faktorial dari angka input.
2. **Child 2:** Menghitung bilangan prima hingga batas tertentu.
3. **Child 3:** Membalik string yang dimasukkan user.

Parent process bertugas:

- Membaca hasil dari tiap child melalui pipe atau shared memory.
- Menunggu setiap child selesai (`waitpid()`).
- Menampilkan PID, exit status, hasil perhitungan, dan waktu eksekusi masing-masing child.

Selain itu, program juga menggunakan **shared memory** (ditunjukkan dari output yang mencetak "Menggunakan variasi shared memory") sesuai aturan variasi NIM.

- **Hasil Uji**

Program berhasil menampilkan:

```
qowi@QOWALIZANI:~$ ./proses_manager
== Process Manager (fork + IPC) ==
Masukkan angka untuk faktorial (child 1): 3
Masukkan batas untuk bilangan prima (child 2): 7
Masukkan string untuk dibalik (child 3): qowi
Masukkan digit terakhir NIM (0-9) untuk variasi shared memory: 2410651083

--- Parent report for child index 1 ---
Child PID: 5897
Exited normally. Exit status = 0
Parent-measured elapsed time for this child: 0.004480 sec
Result (from pipe):
Child1(pid=5897) factorial(3) = 6
Child1_time: 0.000001 sec

--- Parent report for child index 2 ---
Child PID: 5898
Exited normally. Exit status = 0
Parent-measured elapsed time for this child: 0.001361 sec
Result (from pipe):
Child2(pid=5898) primes<7:
Primes = 2: 2 3 5 7
Child2_time: 0.000742 sec

--- Parent report for child index 3 ---
Child PID: 5899
Exited normally. Exit status = 0
Parent-measured elapsed time for this child: 0.001305 sec
Result (from pipe):
Child3(pid=5899) original: 'qowi' reversed: 'iwqo'
Child3_time: 0.000003 sec

Parent: all children handled. Exiting.
qowi@QOWALIZANI:~$
```

Input User

- Angka untuk faktorial: 3
- Batas bilangan prima: 7
- String untuk dibalik: qowi
- Digit terakhir NIM: **ganjil** → menggunakan shared memory

Output Program

1. **Child 1 (PID=5897):**
 - Input: 3
 - Hasil faktorial: $3! = 6$
 - Waktu eksekusi child: 0.000001 sec
 - Waktu terukur oleh parent: 0.004480 sec
 - Status exit: 0 (Exited normally)
2. **Child 2 (PID=5898):**
 - Input: 7
 - Bilangan prima ≤ 7 : 2 3 5 7
 - Waktu eksekusi child: 0.000742 sec
 - Waktu terukur oleh parent: 0.001361 sec
 - Status exit: 0 (Exited normally)
3. **Child 3 (PID=5899):**
 - Input string: "qowi"
 - Hasil string terbalik: "iwqo"
 - Waktu eksekusi child: 0.000003 sec
 - Waktu terukur oleh parent: 0.001305 sec
 - Status exit: 0 (Exited normally)

- **Kesimpulan**

Berdasarkan hasil uji coba, program telah berhasil:

- Membuat 3 child process dengan `fork()`.
- Menggunakan IPC (`pipe` dan `shared memory`).
- Menghindari zombie process dengan `waitpid()`.
- Menghitung faktorial, bilangan prima, dan membalik string sesuai instruksi.
- Menangani perbedaan variasi berdasarkan digit terakhir NIM.

Dengan demikian, implementasi program memenuhi seluruh instruksi praktikum.

TUGAS 2 – FILE MONITOR

- **Pengertian**

Program **File Monitor** dibuat untuk memantau perubahan pada sebuah direktori menggunakan sistem pemantauan event (kemungkinan besar berbasis **inotify** di Linux). Program ini mencatat setiap event yang terjadi ke file log dan, berdasarkan digit terakhir NIM, dapat mengaktifkan simulasi notifikasi email.

Tujuan utama:

- Memahami cara kerja pemantauan file/direktori di level sistem.
- Mencatat event (create, modify, delete) ke file log.
- Mengimplementasikan fitur tambahan (notifikasi email simulasi).

- **Deskripsi Program**

1. Monitoring Directory:

Program memantau direktori `/tmp/monitor_dir` yang dibuat dengan perintah:

- a. `mkdir -p /tmp/monitor_dir`

2. Logging Event:

Semua event yang terdeteksi dicatat ke file `file_monitor.log`.

3. Email Notification Simulation:

- Jika digit terakhir NIM ganjil, maka fitur dummy email notification diaktifkan.
- Setiap event file akan memunculkan pesan simulasi pengiriman email ke `admin@example.com`.

- **Hasil Uji**

Output terminal:

```
qowi@QOWIAIZANI: ~  
qowi@QOWIAIZANI:~$ mkdir -p /tmp/monitor_dir  
qowi@QOWIAIZANI:~$ gcc -o file_monitor file_monitor.c  
qowi@QOWIAIZANI:~$ ./file_monitor /tmp/monitor_dir 7  
[INFO] Monitoring directory: /tmp/monitor_dir  
[INFO] Logging events to: file_monitor.log  
[INFO] Dummy email notification ENABLED (NIM last digit odd)
```

Artinya:

- Direktori berhasil dipantau.
- Logging aktif.
- Email simulasi diaktifkan karena NIM ganjil.

```
qowi@QOWIAIZANI: ~  
qowi@QOWIAIZANI:~$ mkdir -p /tmp/monitor_dir  
qowi@QOWIAIZANI:~$ gcc -o file_monitor file_monitor.c  
qowi@QOWIAIZANI:~$ ./file_monitor /tmp/monitor_dir 7  
[INFO] Monitoring directory: /tmp/monitor_dir  
[INFO] Logging events to: file_monitor.log  
[INFO] Dummy email notification ENABLED (NIM last digit odd)  
[EMAIL SIMULATION] To: admin@example.com | Subject: File Event | Body: [2025-10-16 01:22:00] [MODIFY] [test.txt] [PID:6452]  
[EMAIL SIMULATION] To: admin@example.com | Subject: File Event | Body: [2025-10-16 01:22:00] [MODIFY] [test.txt] [PID:6452]  
[EMAIL SIMULATION] To: admin@example.com | Subject: File Event | Body: [2025-10-16 01:22:13] [MODIFY] [test.txt] [PID:6452]  
[EMAIL SIMULATION] To: admin@example.com | Subject: File Event | Body: [2025-10-16 01:22:21] [DELETE] [test.txt] [PID:6452]
```

- File **test.txt** dalam direktori `/tmp/monitor_dir` dimodifikasi beberapa kali (**MODIFY**).
- Kemudian file tersebut dihapus (**DELETE**).
- Setiap event menghasilkan **simulasi email** yang berisi:
 - **Waktu event** (timestamp).

- **Jenis event** (MODIFY/DELETE).
 - **Nama file** (`test.txt`).
 - **PID proses** yang memicu event (`6452`).
-
- **Kesimpulan**
- Berdasarkan hasil uji coba:
- Program berhasil melakukan monitoring direktori `/tmp/monitor_dir`.
 - Event **MODIFY** dan **DELETE** terdeteksi dengan benar.
 - Logging ke `file_monitor.log` bekerja.
 - Simulasi email notifikasi aktif karena digit terakhir NIM ganjil.

Dengan demikian, program ini sudah memenuhi spesifikasi: **monitoring, logging, dan notifikasi tambahan berbasis NIM**.

TUGAS 3 – CUSTOM SHELL

- **Pengertian**
Program **custom shell** adalah implementasi sederhana dari shell (seperti `bash` atau `sh`) yang dapat menerima perintah dari pengguna, mengeksekusinya menggunakan sistem operasi, serta mendukung fitur **pipes** (`|`), **redirection** (`>`), dan **background process** (`&`).

Program ini bertujuan untuk memahami bagaimana shell berfungsi, dengan memanfaatkan **system calls** seperti `fork()`, `execvp()`, `dup2()`, dan `waitpid()`.

- **Deskripsi Program**

Fitur Utama

1. Eksekusi Perintah Dasar

- Program dapat menjalankan perintah standar Linux seperti `pwd`, `ls`, `cat`, dan lainnya.
- Perintah dieksekusi menggunakan system call seperti `fork()` dan `execvp()`.

2. Input/Output Redirection

- Mendukung redirection output menggunakan `>` (misalnya `echo halo > file.txt`).
- Mendukung redirection input menggunakan `<` (misalnya `cat < file.txt`).

3. Pipeline (|)

- Program dapat menghubungkan output dari satu perintah ke input perintah lain (misalnya `cat file.txt | grep halo`).
- Hal ini diimplementasikan menggunakan system call `pipe()`.

4. Background Process (&)

- Mendukung eksekusi proses di background dengan simbol `&` (misalnya `sleep 5 &`).
- Program akan langsung mengembalikan kontrol ke user dan menampilkan PID proses.

5. Prompt Custom

- Shell menampilkan prompt kustom `shellku>`, sebagai pembeda dengan shell bawaan Linux.

• Hasil Uji

```
qowi@QOWIAIZANI:~$ gcc -o custom_shell custom_shell.c
qowi@QOWIAIZANI:~$ ./custom_shell
shellku> echo "halo dunia" > tes.txt
shellku> cat < tes.txt | grep halo
"halo dunia"
shellku> pwd
/home/qowi
shellku> ls -l | grep .c
-rw-r--r-- 1 qowi qowi      0 Oct 16 02:22 6kb
-rwxr-xr-x 1 qowi qowi 22280 Oct 16 02:33 custom_shell
-rw-r--r-- 1 qowi qowi  7760 Oct 16 02:33 custom_shell.c
-rw-r--r-- 1 qowi qowi  4754 Oct 16 01:46 custom_shell.ck
-rwxr-xr-x 1 qowi qowi 17720 Oct 16 01:21 file_monitor
-rw-r--r-- 1 qowi qowi  3339 Oct 16 01:07 file_monitor.c
-rw-r--r-- 1 qowi qowi  3710 Oct 16 00:55 file_monitor.c.save
-rw-r--r-- 1 qowi qowi   636 Oct 16 01:22 file_monitor.log
-rwxr-xr-x 1 qowi qowi 22136 Oct 16 00:33 proses_manager
-rw-r--r-- 1 qowi qowi 10118 Oct 16 00:41 proses_manager.c
-rw-r--r-- 1 qowi qowi    13 Oct 16 02:33 tes.txt
shellku> echo halo > output.txt
shellku> cat output.txt
halo
shellku> sleep 5 &
[latar belakang] pid 6979
```

• Membuat file teks dan membaca isinya:

```
shellku> echo "halo dunia" > tes.txt
shellku> cat < tes.txt | grep halo
"halo dunia"
```

- Menampilkan direktori saat ini:

```
shellku> pwd
/home/qowi
```

- Pipeline untuk filter file berekstensi .c:

```
shellku> ls -l | grep .c
```

- Menjalankan proses di background:

```
shellku> sleep 5 &
[latar belakang] pid 6979
```

- **Kesimpulan**

Program **Custom Shell** ini merupakan simulasi dari shell Linux dengan fitur dasar. Walaupun sederhana, ia mampu memperlihatkan konsep fundamental dari **sistem operasi**, meliputi:

- Manajemen proses (fork, exec, wait).
- Manajemen file descriptor (dup2 untuk redirection).
- Komunikasi antar proses (pipe).
- Pengelolaan proses background.

TUGAS 4 – SYSTEM INFORMATION TOOL

- **Pengertian**

Program **System Information Tool** adalah aplikasi berbasis C yang digunakan untuk mengumpulkan dan menampilkan informasi sistem Linux secara langsung dari **file /proc** dan system call **statvfs()**. Program ini mendemonstrasikan bagaimana data terkait CPU, memori, disk, proses, dan jaringan dapat diakses tanpa bantuan library eksternal tingkat tinggi.

Tujuan program ini adalah agar mahasiswa memahami bagaimana sistem operasi **menyediakan informasi status kernel dan hardware** melalui interface **/proc** dan system call standar.

- **Deskripsi Program**

Program membaca informasi dari sumber berikut:

1. **CPU Info** → dari file **/proc/cpuinfo**

- Menampilkan model CPU, jumlah logical cores, dan frekuensi CPU.
- 2. **Memory Info** → dari file `/proc/meminfo`
 - Menampilkan total, free, buffer, cached, dan perkiraan memori terpakai.
- 3. **Disk Usage** → dengan system call `statvfs("/")`
 - Menampilkan kapasitas total, digunakan, free, dan available (untuk non-root).
- 4. **Running Processes** → dari `/proc/[PID]/status`
 - Melakukan iterasi pada direktori `/proc` untuk mencari proses aktif.
 - Menampilkan **PID, Nama Proses, dan State**.
- 5. **Network Interfaces** → dari `/proc/net/dev`
 - Menampilkan statistik bytes yang diterima (RX) dan dikirim (TX) dari setiap interface.

• Hasil Uji

```

qowi@QOWIAIZANI: ~$ gcc -o system_info system_info.c
qowi@QOWIAIZANI: ~$ ./system_info

SYSTEM INFORMATION TOOL
Gathering info from /proc and statvfs()

=====
CPU INFO
=====
Model       : 13th Gen Intel(R) Core(TM) i7-13620H
Logical CPUs : 2 (logical)
Frequency   : 2918.40 MHz

=====
MEMORY INFO
=====
Total       : 7792.18 MB
Free        : 7145.21 MB
Buffers     : 3.64 MB
Cached (incl. SReclaimable) : 219.98 MB
Used (approx) : 423.36 MB

=====
DISK USAGE (root filesystem)
=====
Total       : 1006.85 GB
Used        : 1.72 GB
Free        : 1005.13 GB
Available (non-root) : 953.91 GB

=====
RUNNING PROCESSES (PID | NAME | STATE)
=====
PID    NAME                STATE
1      systemd
2      init-systemd(Ub
8      init
54     systemd-journal
75     systemd-udev
83     systemd-network
275    snapfuse
279    snapfuse
280    snapfuse
287    systemd-resolve

```

```
=====
RUNNING PROCESSES (PID | NAME | STATE)
=====
PID      NAME                STATE
1        systemd
2        init-systemd(Ub
8        init
54       systemd-journal
75       systemd-udev
83       systemd-network
275      snapfuse
279      snapfuse
280      snapfuse
287      systemd-resolve
290      accounts-daemon
291      dbus-daemon
297      networkd-dispat
298      polkitd
299      rsyslogd
300      snapd
306      systemd-logind
309      udisksd
327      ModemManager
347      cron
351      atd
363      unattended-upgr
364      agetty
368      agetty
577      login
638      systemd
639      (sd-pam)
653      bash
6991     SessionLeader
6992     Relay(6995)
6995     bash
7031     system_info

=====
NETWORK INTERFACES (RX bytes | TX bytes)
=====
Interface  RX bytes  TX bytes
lo         12246     12246

Done.
```

1. CPU INFO

Model : 13th Gen Intel(R) Core(TM) i7-13620H
Logical CPUs : 2 (logical)
Frequency : 2918.40 MHz

✓ Berhasil menampilkan detail prosesor.

2. MEMORY INFO

Total : 7792.18 MB
Free : 7145.21 MB
Buffers : 3.64 MB
Cached : 219.98 MB
Used : 423.36 MB

✓ Informasi memori berhasil ditarik dari /proc/meminfo.

3. DISK USAGE

Total : 1006.85 GB
Used : 1.72 GB
Free : 1005.13 GB
Available: 953.91 GB

✓ Informasi kapasitas filesystem root (/) berhasil ditampilkan.

4. RUNNING PROCESSES

Contoh output:

PID	NAME	STATE
1	systemd	S
2	init	S
54	systemd-journal	S
275	snapfuse	S
653	bash	S
7031	system_info	R

✓ Program dapat membaca daftar proses aktif dengan PID, nama, dan status.

5. NETWORK INTERFACES

Interface	RX bytes	TX bytes
lo	12246	12246

✓ Statistik jaringan interface loopback berhasil ditampilkan.

- **Kesimpulan**

- Program berhasil memanfaatkan **system call** (`open`, `read`, `statvfs`) dan **file virtual di /proc** untuk menampilkan informasi sistem.
- Semua komponen (CPU, memori, disk, proses, dan jaringan) tampil sesuai dengan status aktual sistem Linux.
- Tool ini bisa dikembangkan lebih lanjut dengan menambahkan:
 - Monitoring real-time (dengan `sleep` dan `refresh`).
 - Filter proses berdasarkan user tertentu.
 - Dukungan multi-interface jaringan.